



UNIVERSITAT
POLITÈCNICA
DE VALÈNCIA



Escola Tècnica
Superior d'Enginyeria
Informàtica

Escola Tècnica Superior d'Enginyeria Informàtica
Universitat Politècnica de València

Manual de Programación

Trabajo Fin de Grado

Grado en Informática Industrial y Robótica

Autor: Lourdes Francés Llimerá

Tutor: Juan Francisco Blanes Noguera

2025-2026

Tabla de contenidos

Tabla de contenidos.....	2
Índice de código.....	4
Índice de tablas.....	5
1. Estructura del proyecto.....	6
2. Dependencias y entorno de ejecución	6
3. config.py – configuración global	7
3.1. Conmutadores de entorno.....	7
3.2. Resolución de rutas.....	7
3.3. Endpoints y modelos del LLM	7
3.4. Parámetros de movimiento.....	7
3.5. Geometría de la HMI	8
3.6. Mapa semántico (KNOWN_LOCATIONS)	8
3.7. SYSTEM_PROMPT.....	8
4. i18n.py – internacionalización	8
5. agent_logic.py – agente y control de ROS 2.....	9
5.1. Supresión de ruido (antes de los imports)	9
5.2. Importaciones	9
5.3. Esquemas de entrada (Pydantic)	10
5.4. Modelo de concurrencia: las tres primitivas compartidas.....	10
5.5. Patrón común de las herramientas de movimiento.....	11
5.6. SpinTool (spin_robot)	12
5.7. MoveTool (move_robot)	12
5.8. SequenceTool (execute_sequence).....	12
5.9. NavigationTool (navigate_to_location)	12
5.10. NavigationSequenceTool (navigate_sequence)	13
5.11. GestureTool (negation_gesture).....	13
5.12. StopTool (stop_robot).....	13
5.13. Funciones utilitarias.....	14
5.14. Punto de entrada main()	14
6. laric_interface.py - interfaz HMI	15
6.1. Integración de ROS 2 en el bucle de Qt.....	15
6.2. Publicadores y suscriptores	15
6.3. Push-to-Talk y procesamiento de audio (STT)	15
6.4. Salida por voz (TTS)	16

6.5. Entrada de texto.....	16
6.6. Log seguro entre hilos.....	16
6.7. Selector de idioma.....	16
6.8. Parada de emergencia.....	16
6.9. Cierre de la ventana	17
6.10. Punto de entrada (main).....	17
7. Scripts de arranque y parada	17
8. Parámetros de Nav2 (config/laric_nav2_params.yaml)	17
9. El fork de RAI y los errores en nav2.py	18
9.1. Estado de la navegación no reiniciado entre misiones	18
9.2. GoalStatus descartado en el resultado.....	18
10. Internacionalización: catálogos	19

Índice de código

Código 1. LARIC workspace.	6
Código 2. Conmutadores de entorno.....	7
Código 3. Internacionalización.....	9
Código 4. Esquema de entrada de SpinInput.....	10
Código 5. Patrón de herramientas de movimiento (SpinTool._run).	12
Código 6. StopTool.	13
Código 7. Construcción LLM y agente.....	15
Código 8. Reinicio del estado de la navegación (RAI).....	18
Código 9. Obtención del GoalStatus (RAI).....	19

Índice de tablas

Tabla 1. Parámetros de movimiento.....	8
--	---

1. Estructura del proyecto

Este manual describe el código de LARIC de forma que pueda entenderse sin haberlo visto antes: recorre cada archivo, cada clase y cada función relevante, explicando no solo qué hace sino por qué está hecho así. Para una visión conceptual de las tecnologías (ROS 2, Nav2, LLM, RAI, etc.) y de cómo se interconectan, consultar el documento “Documentación del Proyecto”, este manual se centra en la implementación.

Todo el código vive en el workspace de ROS 2 `laric_ws`. El paquete principal es `laric_core`. El sistema son dos procesos Python que se comunican por tópicos ROS 2: el agente (`agent_logic.py`) y la interfaz (`laric_interface.py`), apoyados en dos módulos compartidos (`config.py` e `i18n.py`).

```
laric_ws/
  requirements.txt          # dependencias Python de laric_env
  scripts/                 # arranque y parada (sim y real)
    start_simulation.sh    stop_simulation.sh
    start_real.sh         stop_real.sh
  locales/                 # catalogos gettext es/en + laric.pot
  maps/                    # mapas (tb3_house, laboratory)
  src/laric_core/
    package.xml setup.py resource/ # metadatos del paquete ROS 2
    config/laric_nav2_params.yaml  # parametros de Nav2 (simulacion)
    laric_core/
      agent_logic.py          # agente, herramientas, control ROS 2
      laric_interface.py      # HMI PyQt5 + Push-to-Talk
      config.py               # configuracion (API, mapa, prompt, rutas)
      i18n.py                 # traductor gettext (_ / set_language)
```

Código 1. LARIC workspace.

El paquete se compila con `colcon` (build system de ROS 2). `setup.py` declara el paquete `laric_core` y, en `data_files`, instala el `package.xml` y el YAML de Nav2. No se declaran `console_scripts`: los procesos se lanzan con `python3` desde los scripts de arranque, no como ejecutables de ROS 2.

2. Dependencias y entorno de ejecución

Las dependencias de Python están en `requirements.txt` y se instalan en un entorno virtual (`laric_env`) creado con `--system-site-packages`, de modo que el entorno aísle las librerías de IA pero conserve el acceso a la instalación base de ROS 2. Los grupos principales son:

- **Ecosistema LangChain:** `langchain`, `langchain-core`, `langchain-community`, `langchain-groq`, `langchain-ollama` (y otros adaptadores), `langgraph`. Orquestan el agente y la conexión con los dos backends del LLM.
- **Audio e interfaz:** `numpy` (<2.0), `SpeechRecognition` (Google STT), `sounddevice` (captura de micrófono), `scipy` (escritura WAV), `edge-tts` (síntesis de voz neuronal para la salida hablada) y `PyQt5` (GUI).

Aparte de requirements.txt, LARIC depende de un fork modificado de RAI (github.com/lourdesfl29-maker/rai), que se instala por separado y es imprescindible (véase la sección 9). La instalación completa de ambos entornos (simulación y robot real) está en el “Manual de Configuración”.

3. config.py - configuración global

Módulo sin funciones: solo declara valores, importados por agent_logic.py y laric_interface.py. Centraliza todo lo configurable para no dispersar las variables por el código. Sus bloques son:

3.1. Conmutadores de entorno

- **is_from_lab (bool):** True usa el Ollama del ai2 (vía principal); False usa Groq en la nube (respaldo desde fuera del laboratorio).
- **is_real_robot (bool):** False = simulación (Gazebo); True = robot real (Husarion). Selecciona el conjunto de ubicaciones y el script de apagado. Es independiente de is_from_lab.

```
is_from_lab:    bool = True      # True: Ollama ai2;  False: Groq nube
is_real_robot: bool = False     # False: simulacion; True: ROSbot real
```

Código 2. Conmutadores de entorno.

3.2. Resolución de rutas

BASE_DIR es la carpeta de config.py. PROJECT_ROOT sube tres niveles hasta la raíz del workspace. SHUTDOWN_SCRIPT_PATH apunta a stop_real.sh o stop_simulation.sh según is_real_robot. Calcular las rutas a partir de la ubicación del archivo evita escribir rutas absolutas a mano.

3.3. Endpoints y modelos del LLM

- **URL:** Endpoint del servidor Ollama del ai2, accesible solo desde la red de la UPV. Se define en config.py.
- **MODEL:** Nombre del modelo en el servidor Ollama (llama3.3:70b).
- **GROQ_API_KEY:** Clave de Groq, leída de la variable de entorno.
- **GROQ_MODEL_70B:** Modelo de Groq (llama-3.3-70b-versatile).

3.4. Parámetros de movimiento

Reúne en un solo sitio los valores que ajustan el movimiento, para poder afinarlos sin tocar agent_logic.py.

Parámetro	Valor	Significado
SPEED_SLOW	0.15 m/s	Velocidad lineal lenta.
SPEED_NORMAL_FORWARD	0.3 m/s	Velocidad de avance por defecto.
SPEED_NORMAL_BACK	0.2 m/s	Velocidad de retroceso por defecto.
SPEED_FAST	0.6 m/s	Velocidad lineal rápida.

SPIN_SPEED_RAD_S	0.5 rad/s	Velocidad de giro estimada (dimensiona el time_allowance).
POLL_INTERVAL_S	0.2 s	Sondeo de los movimientos primitivos.
NAV_POLL_INTERVAL_S	0.5 s	Sondeo de la navegación.
NAV_TIMEOUT_S	180 s	Tiempo máximo de una navegación (3 min).
INITIALPOSE_GRACE_S	5 s	Ventana de gracia de la localización al arrancar.

Tabla 1. Parámetros de movimiento.

Además, tres márgenes (SPIN_ALLOWANCE_BUFFER_S, SPIN_ALLOWANCE_MIN_S y MOVE_ALLOWANCE_BUFFER_S) acolchan el time_allowance que Nav2 concede a cada movimiento, para que no aborte uno legítimo por creerlo demasiado lento.

3.5. Geometría de la HMI

HMI_GEOMETRY (x, y, ancho, alto) de la ventana, sobrescribible con la variable de entorno LARIC_HMI_GEOMETRY sin editar el código.

3.6. Mapa semántico (KNOWN_LOCATIONS)

Diccionario {nombre: {x, y, yaw}} en coordenadas del mapa. Hay dos conjuntos:

- **KNOWN_LOCATIONS_SIM:** 8 estancias de turtlebot3_house.
- **KNOWN_LOCATIONS_REAL:** 13 puntos de la planta del ai2, obtenidos navegando el robot real.

La variable activa KNOWN_LOCATIONS se elige según is_real_robot. Es la base de conocimiento que traduce un nombre ("dining_room") a coordenadas para la navegación.

3.7. SYSTEM_PROMPT

Define el comportamiento del LLM, siendo la pieza más delicada del sistema. Contiene tres cosas:

- **La regla de idioma:** Responder siempre en el idioma del usuario.
- **Las reglas de selección de cada herramienta,** con sus convenciones de signo y ejemplos de conversión.
- **Cinco reglas críticas de ejecución:** Una sola herramienta por turno, la asincronía de Nav2, no auto-cancelarse, cómo evaluar el resultado y paradas solo explícitas.

La lista de ubicaciones se inyecta en el texto con replace('__LOCATIONS_LIST__', ...). Un detalle importante es que los nombres de herramienta del prompt deben coincidir exactamente con el atributo name de cada BaseTool en agent_logic.py.

4. i18n.py - internacionalización

Fuente única de la traducción. Expone tres elementos:

- **set_language(lang):** Activa el catálogo gettext de un idioma.
- **get_language():** Devuelve el código activo.
- **_(message):** Traduce el texto al idioma activo (devuelve el original si no hay traducción).

La clave de diseño está en `_()`. Cada vez que se llama, mira qué catálogo de traducción está activo en ese momento y traduce con él. La alternativa sería guardar el traductor una sola vez, al importar el módulo. El problema de esa alternativa es que, si el usuario cambia de idioma después, los módulos ya importados seguirían traduciendo al idioma viejo. Al resolverlo en cada llamada, un cambio de idioma surte efecto al instante en todo el sistema.

Esto funciona porque `_translator` es una variable de nivel de módulo: cuando `set_language` la reasigna, la siguiente llamada a `_()` ya usa el nuevo catálogo. Además, el catálogo se carga con `fallback=True`, así que una traducción que falte no provoca un error. Simplemente se devuelve el texto original, en inglés.

```
def _(message: str) -> str:
    # Resuelve el catalogo ACTIVO en cada llamada (no en el import):
    return _translator.gettext(message)

def set_language(lang: str) -> None:
    global _current_language, _translator
    _current_language = lang
    _translator = gettext.translation(
        domain=DOMAIN, localedir=LOCALE_DIR,
        languages=[lang], fallback=True)
```

Código 3. Internacionalización.

5. agent_logic.py - agente y control de ROS 2

Es el archivo central: define los esquemas de entrada, las seis herramientas, las utilidades y el punto de entrada `main()`.

5.1. Supresión de ruido (antes de los imports)

Lo primero del archivo, antes de importar las librerías de terceros, instala filtros que silencian tres avisos inofensivos pero ruidosos:

- El `SyntaxWarning` de `pydub`.
- Los avisos de deprecación de `langgraph`.
- Los `WARNING` de "rai_interfaces is not installed" del logger raíz. Estos se filtran con una subclase de `logging.Filter` (`_RAIInterfacesNoise`).

El orden importa. Los filtros deben instalarse antes de importar las librerías que generan esos avisos. Si se hiciera después, el aviso ya se habría emitido.

5.2. Importaciones

Tras la supresión de ruido se importan, por grupos:

- **ROS 2 / Nav2:** GoalStatus (action_msgs) y las definiciones de acción BackUp, DriveOnHeading y Spin (nav2_msgs).
- **LangChain:** AgentExecutor, create_tool_calling_agent, BaseTool, ChatPromptTemplate, ChatGroq y ChatOllama.
- **RAI:** ROS2Context, ROS2Message, ROS2Connector y las herramientas de Nav2 del fork (NavigateToPoseTool, etc.).
- **Del proyecto:** Los valores de config.py, y el traductor _ y set_language de i18n.

5.3. Esquemas de entrada (Pydantic)

Cada herramienta valida sus entradas con un modelo Pydantic. Lo esencial es que las descripciones de los campos (Field(description=...)) las lee el propio LLM para inferir los parámetros: son, de hecho, parte del prompt efectivo.

- **SpinInput:** angle en grados (positivo = izquierda, negativo = derecha), con ejemplos de conversión de "media vuelta", "ángulo recto", etc.
- **MoveInput:** distance (metros con signo) y speed en m/s (0.0 deja que la herramienta aplique el valor por defecto).
- **SequenceStep / SequenceInput:** Una lista ordenada de pasos, cada uno con action ('move' o 'spin'), value y speed.
- **NavigationInput:** location_name (debe coincidir con el mapa semántico) y, alternativamente, target_x / target_y / target_yaw para coordenadas crudas.
- **TriggerInput:** Un único campo execute=True, para herramientas sin parámetros (StopTool), porque LangChain exige que todo esquema declare al menos un campo.
- **GestureInput:** reason, breve explicación de por qué no se puede ejecutar la petición.

```
class SpinInput(BaseModel):
    angle: float = Field(
        default=0.0,
        description=( # el LLM LEE esta descripcion para inferir el valor
            "Target rotation angle in degrees. "
            "Positive = left (CCW). Negative = right (CW). "
            "media vuelta -> -180.0, angulo recto -> -90.0."
        ))
```

Código 4. Esquema de entrada de SpinInput.

5.4. Modelo de concurrencia: las tres primitivas compartidas

Las herramientas de movimiento se coordinan mediante tres primitivas de threading, creadas en main() e inyectadas en cada herramienta por construcción. Entenderlas es clave para entender todo el archivo:

- **localised_event (threading.Event):** Se activa cuando el operador fija la pose inicial (callback on_initial_pose). Es prerequisite de todo movimiento: si no está activo, la herramienta devuelve "ACTION FAILED. The robot is not localised."

- **motion_lock (threading.RLock, reentrante):** Cada herramienta de movimiento intenta adquirirlo de forma no bloqueante al entrar. Si está ocupado, rechaza el comando (solo un movimiento a la vez). Es un RLock, no un Lock simple, porque SequenceTool lo adquiere una vez para toda la secuencia y luego sus sub-tools lo vuelven a adquirir de forma reentrante en cada paso.
- **abort_event (threading.Event):** StopTool lo activa (set) y los bucles de sondeo lo consultan (is_set) para cancelar la acción en curso. Cada comando standalone lo limpia (clear) al empezar, de modo que una cancelación anterior no se cuele en el siguiente comando.

5.5. Patrón común de las herramientas de movimiento

SpinTool, MoveTool y NavigationTool comparten una estructura. Conocerla evita repetir la explicación en cada una:

- **Paso 0 - precondiciones:** Comprobar localised_event (si no, ACTION FAILED) y adquirir motion_lock de forma no bloqueante (si está ocupado, rechazar).
- **Limpieza de abort_event:** Si standalone, abort_event.clear() (parte clave de la seguridad ante cancelaciones previas).
- **Anuncio:** Publicar en /robot_feedback (vía _publish_feedback) lo que va a hacer, traducido con _().
- **Construcción del goal:** Armar el diccionario del goal, incluido un time_allowance calculado por _compute_spin_allowance o _compute_move_allowance.
- **Envío y sondeo:** Enviar con connector.start_action y entrar en un bucle while que, cada POLL_INTERVAL_S, comprueba si la acción terminó (action_done), si se pidió abortar (abort_event) o si se agotó el timeout.
- **Cierre:** En un finally, liberar motion_lock siempre, y devolver una cadena con prefijo SUCCESS / ACTION FAILED / ACTION ABORTED / ERROR.

La acción se completa mediante un callback on_done que lee result.result().status y rellena un result_container local, que el bucle de sondeo recoge. Las cadenas de retorno acaban en "DO NOT CALL ANY OTHER TOOLS." porque la regla 4 del prompt las usa para que el LLM no encadene acciones ni explique el resultado.

Esqueleto del patrón (simplificado a partir de SpinTool._run):

```
# 0. Precondiciones (comunes a las herramientas de movimiento)
if not self.localised_event.is_set():
    return "ACTION FAILED. The robot is not localised. ..."
if not self.motion_lock.acquire(blocking=False):
    return "ACTION FAILED. Another action is in progress. ..."
try:
    if standalone:
        self.abort_event.clear() # no heredar un stop anterior
    handle = self.connector.start_action(
        action_data=ROS2Message(payload=goal_dict),
        target="/spin", msg_type="nav2_msgs/action/Spin",
        on_feedback=_on_feedback, on_done=_on_done)
    # Bucle de sondeo: fin de la acción, abort o timeout
    while not action_done.is_set() and elapsed < timeout:
```

```

    if self.abort_event.is_set():
        self.connector.terminate_action(handle)
        return "ACTION ABORTED. The user stopped the robot. ..."
    time.sleep(poll_interval)
    elapsed += poll_interval
finally:
    self.motion_lock.release()          # liberar SIEMPRE

```

Código 5. Patrón de herramientas de movimiento (SpinTool._run).

5.6. SpinTool (spin_robot)

Giro en el sitio mediante el behavior plugin /spin (nav2_msgs/action/Spin). Convierte el ángulo a radianes, calcula el time_allowance con _compute_spin_allowance y envía el goal target_yaw. Sigue el patrón común. El signo del ángulo respeta la convención del prompt (positivo = izquierda).

5.7. MoveTool (move_robot)

Traslación lineal. Según el signo de distance elige el action server: /drive_on_heading (avance, nav2_msgs/action/DriveOnHeading) o /backup (retroceso, nav2_msgs/action/BackUp). Si speed es 0.0 aplica el valor por defecto (SPEED_NORMAL_FWD o SPEED_NORMAL_BACK). El resto sigue el patrón común.

5.8. SequenceTool (execute_sequence)

Ejecuta varios movimientos en orden. Su funcionamiento:

- En __init__ crea instancias internas de SpinTool y MoveTool, que comparten las mismas primitivas. Así, un StopTool a mitad de secuencia las afecta a todas.
- Adquiere motion_lock una sola vez para toda la secuencia y limpia abort_event una sola vez.
- Despacha cada paso llamando a la sub-tool con standalone=False, para que no vuelva a adquirir el lock en exclusiva ni vuelva a limpiar abort_event.
- Comprueba abort_event antes de cada paso.
- Si un paso devuelve ERROR o ABORTED, detiene la secuencia y lo propaga hacia arriba.
- Entre pasos hace una breve pausa de estabilización.

5.9. NavigationTool (navigate_to_location)

Navegación autónoma a un destino. Sus pasos:

- En __init__ crea las herramientas de RAI: NavigateToPoseTool, GetNavigateToPoseFeedbackTool y GetNavigateToPoseResultTool.
- Resuelve el destino. Las coordenadas explícitas tienen prioridad sobre el nombre. Si es un nombre, lo busca en KNOWN_LOCATIONS.
- Envía el goal con _rai_nav_tool._run(x, y, z, yaw).
- Entra en un bucle de sondeo (cada NAV_POLL_INTERVAL_S, hasta NAV_TIMEOUT_S = 3 min) que consulta _rai_nav_result_tool._run().

- Compara el GoalStatus devuelto: SUCCEEDED es llegada, ABORTED o CANCELED es fallo.

Este sondeo es justamente lo que depende de las correcciones del fork de RAI (véase la sección 9).

Acepta además el parámetro standalone (por defecto True): NavigationSequenceTool lo invoca con standalone=False para no reiniciar abort_event en cada tramo de la patrulla.

5.10. NavigationSequenceTool (navigate_sequence)

Patrulla de varias ubicaciones. Recibe una lista ordenada de nombres de ubicación y la recorre punto por punto. En __init__ crea una instancia interna de NavigationTool con la que comparte las primitivas. Valida toda la ruta antes de empezar (rechaza ubicaciones desconocidas sin mover el robot), adquiere motion_lock y limpia abort_event una sola vez para toda la ruta. Para cada destino llama a NavigationTool._run(location_name=..., standalone=False); al llegar espera PATROL_DWELL_S segundos y continúa. Un stop_robot a mitad de ruta la detiene limpiamente, entre tramos o durante uno.

5.11. GestureTool (negation_gesture)

Gesto físico de negación ante peticiones imposibles. No usa Nav2: publica directamente en /cmd_vel (geometry_msgs/Twist) una oscilación de velocidad angular alternada ([1.0, -1.0, 1.0, -1.0]) y luego un Twist de ceros. Adquiere motion_lock (para no solapar otra escritura en /cmd_vel) pero no comprueba localised_event, por tratarse de una señal social breve y no de un desplazamiento. Es interrumpible: comprueba abort_event entre oscilaciones.

5.12. StopTool (stop_robot)

Parada inmediata. Es especial porque no adquiere motion_lock: su misión es interrumpir un movimiento que ya lo tiene. Hace tres cosas:

- abort_event.set(), para que todos los bucles de sondeo terminen su acción.
- Cancela el goal de Nav2 activo con CancelNavigateToPoseTool de RAI.
- Publica un Twist de ceros en /cmd_vel como respaldo inmediato.

Los pasos 2 y 3 van en try/except, porque no son fatales si no hay ningún goal activo.

```
def _run(self, **kwargs) -> str:
    self.abort_event.set()          # 1. corta los bucles de sondeo
    try:
        self._cancel_nav_tool._run() # 2. cancela el goal de Nav2
    except Exception:
        pass
    self.connector.send_message(     # 3. Twist de ceros (respaldo)
        ROS2Message(payload={"linear": {"x": 0.0, "y": 0.0, "z": 0.0},
                              "angular": {"x": 0.0, "y": 0.0, "z": 0.0}}),
        target="/cmd_vel", msg_type="geometry_msgs/msg/Twist")
    return "SUCCESS. Stop activated. ..."
```

Código 6. StopTool.

5.13. Funciones utilitarias

- **`_publish_feedback(connector, message)`:** Publica un `std_msgs/String` en `/robot_feedback` (toda la realimentación del agente pasa por aquí). Si se le pasa `voice`, publica además una frase corta en `/laric_voice` para que la HMI la lea en voz alta.
- **`_compute_spin_allowance(yaw)`:** Estima la duración del giro ($|\text{yaw}|$ / velocidad) y le suma un margen, con un mínimo, para el `time_allowance` de Nav2.
- **`_compute_move_allowance(distancia, velocidad)`:** Estima la duración del avance ($\text{distancia} / \text{velocidad}$) más un margen.

Los `time_allowance` generosos evitan que Nav2 aborte un movimiento legítimo por creerlo demasiado lento.

5.14. Punto de entrada `main()`

Decorado con `@ROS2Context()` (RAI inicializa y limpia `rcipy`). Sus pasos:

1. Crea el `ROS2Connector` y espera 3 s para que sus publicadores se registren.
2. Crea las tres primitivas de sincronización y `_localised_ready_at = ahora + INITIALPOSE_GRACE_S` (ventana de gracia de la localización).
3. Instancia las seis herramientas inyectándoles `connector` y primitivas (inyección de dependencias).
4. Conecta con el LLM. Si `is_from_lab`, usa `ChatOllama(temperature=0, model=MODEL, base_url=URL)`. Si no, usa `ChatGroq(temperature=0, model=GROQ_MODEL_70B, api_key=GROQ_API_KEY)` y avisa si falta la clave.
5. Construye el prompt (`system + human + placeholder de agent_scratchpad`), el agente (`create_tool_calling_agent`) y el `AgentExecutor(verbose, max_iterations=6, handle_parsing_errors)`.
6. Registra los callbacks de `/from_human`, `/initialpose`, `/emergency_stop` y `/language_changed`, y entra en un bucle `keep-alive`.

Los cuatro callbacks registrados hacen lo siguiente:

- **`on_human_input`:** Publica lo que ha oído y lanza el razonamiento en un hilo `daemon` (`think()`) para no bloquear el spin de ROS 2. `think()` invoca el `AgentExecutor`. Si la respuesta no es una cadena interna (no empieza por `SUCCESS/ERROR` ni contiene "DO NOT CALL..."), la publica como mensaje conversacional, con los saltos de línea convertidos a `<br>`.
- **`on_initial_pose`:** Ignora las poses recibidas antes de `_localised_ready_at` (poses retenidas por DDS). Pasada la ventana de gracia, activa `localised_event`.
- **`on_emergency_stop`:** Ejecuta `StopTool._run()` directamente, evitando el LLM.
- **`on_language_changed`:** Llama a `set_language` con el nuevo código.

Construcción del LLM y del agente (paso central de `main`):

```
# Seleccion del backend del LLM (mismo modelo de 70B)
```

```

if is_from_lab:
    llm = ChatOllama(temperature=0, model=MODEL, base_url=URL)
else:
    llm = ChatGroq(temperature=0, model=GROQ_MODEL_70B,
                  api_key=GROQ_API_KEY)

prompt = ChatPromptTemplate.from_messages([
    ("system", SYSTEM_PROMPT),
    ("human", "{input}"),
    ("placeholder", "{agent_scratchpad}")])
agent = create_tool_calling_agent(llm, tools, prompt)
agent_executor = AgentExecutor(agent=agent, tools=tools,
                               verbose=True, max_iterations=6, handle_parsing_errors=True)

```

Código 7. Construcción LLM y agente.

6. laric_interface.py - interfaz HMI

La clase InterfaceNode hereda a la vez de rclpy.node.Node y de PyQt5.QWidget. Combina así, en un único objeto, un nodo de ROS 2 y una ventana gráfica.

6.1. Integración de ROS 2 en el bucle de Qt

Cada framework tiene su propio bucle de eventos. Para conciliarlos, un QTimer periódico (ROS_SPIN_INTERVAL_MS = 20 ms) ejecuta _spin_ros, que llama a rclpy.spin_once(self, timeout_sec=0). Así ROS 2 procesa mensajes sin bloquear la GUI. Antes de importar PyQt5 se fuerza el backend X11 (QT_QPA_PLATFORM=xcb) para evitar problemas con Wayland en Ubuntu 24.

6.2. Publicadores y suscriptores

Publica en /from_human (texto del usuario), /cmd_vel (Twist de la parada directa), /emergency_stop (Bool) y /language_changed (código de idioma). Se suscribe a /robot_feedback, cuyo callback vuelca cada mensaje en el log y a /laric_voice (frases que se leen en voz alta).

6.3. Push-to-Talk y procesamiento de audio (STT)

La entrada por voz funciona como un Push-to-Talk: se graba mientras se mantiene pulsada la barra espaciadora. Los métodos implicados son:

- **keyPressEvent / keyReleaseEvent:** Detectan la barra espaciadora. Al pulsarla empieza a grabar. Al soltarla esperan un instante antes de parar (un debounce de PTT_DEBOUNCE_MS = 150 ms), para que un rebote de la tecla no corte la grabación. Si se está escribiendo en el campo de texto, el espacio se trata como un espacio normal.
- **_start_recording:** Abre el micrófono (un sounddevice.InputStream a 16 kHz, mono) y cambia el aspecto de la interfaz para indicar que está grabando. Mientras abre el micrófono, silencia con _suppress_stderr los mensajes de error de los drivers de audio (ALSA/PortAudio), que son muy ruidosos.
- **_audio_callback:** Va guardando los fragmentos de audio según llegan del micrófono.

- **`_stop_recording`:** Cierra el micrófono y lanza la transcripción en un hilo aparte (`_process_audio`). Usar un hilo aparte evita que la interfaz se congele, porque transcribir implica una llamada por Internet.
- **`_process_audio`:** Convierte el audio grabado en texto. Primero une los fragmentos. Si la grabación dura menos de `MIN_AUDIO_DURATION_S` (0.2 s) la descarta, para ignorar pulsaciones accidentales. Después convierte el audio al formato WAV que espera el motor de voz y lo envía a la API de Google (`recognizer.recognize_google`) en el idioma activo. Por último, publica el texto reconocido (en minúsculas) en `/from_human`. Trata por separado dos errores: que no se detecte voz y que la API no responda.

6.4. Salida por voz (TTS)

Además del log en pantalla, la HMI reproduce por voz una versión corta de cada mensaje. La clase `VoiceSpeaker` sintetiza el texto con `edge-tts` (voces neuronales de Microsoft, más naturales que otras alternativas) a un MP3 temporal y lo reproduce con el primer reproductor disponible del sistema (`ffplay`, `mpg123`, `mpg321` o `cvlc`). La HMI se suscribe a `/laric_voice` y usa dos hilos encadenados: uno sintetiza por adelantado y otro reproduce en orden, sin bloquear la interfaz; el idioma se mantiene sincronizado con el desplegable de selección de idioma. La voz es interrumpible: una parada vacía las colas y silencia la reproducción en curso. El audio se reproduce en el PC (el ROSbot 3 no tiene altavoz).

6.5. Entrada de texto

`_send_text_command` es la entrada por teclado, alternativa a la voz. Toma el texto del campo y lo registra en el log, lo publica en minúsculas en `/from_human` y limpia el campo.

6.6. Log seguro entre hilos

La interfaz de Qt solo puede actualizarse desde el hilo principal. Pero el audio y la realimentación del robot llegan desde hilos secundarios. Si esos hilos escribieran directamente en el log, podrían provocar fallos (lo que se conoce como condición de carrera). Para evitarlo, `add_log` no toca el log directamente. En su lugar emite una señal de Qt (un `pyqtSignal` llamado `log_signal`) conectada al método `update_log_slot`. Qt entrega siempre esa señal en el hilo principal, así que la escritura en el log (una línea HTML con color) ocurre de forma segura.

6.7. Selector de idioma

`_update_language` gestiona el cambio de idioma desde el desplegable. Traduce la opción elegida a dos códigos: uno para el reconocimiento de voz (`en-US` o `es-ES`) y otro para `gettext` (`en` o `es`). Llama a `set_language` para cambiar el idioma en la propia interfaz y retraduce las etiquetas fijas. Por último, publica el nuevo código en `/language_changed`, para que el agente (que es otro proceso) sincronice su idioma.

6.8. Parada de emergencia

`_emergency_stop` se activa con el botón rojo o con Ctrl+E (registrado como un QShortcut). Hace dos cosas a la vez, evitando por completo el LLM:

- Publica un Twist de ceros en `/cmd_vel`, para frenar el robot de inmediato.
- Publica un Bool True en `/emergency_stop`, para que el agente cancele los goals de Nav2.

6.9. Cierre de la ventana

`closeEvent` intercepta el cierre de la ventana (la X). Antes de cerrar, lanza el script de apagado (`SHUTDOWN_SCRIPT_PATH`, resuelto en `config.py`) en una sesión separada, para detener todos los nodos y procesos del sistema. También borra el SVG temporal de la flecha del desplegable, que se había creado al construir la interfaz.

6.10. Punto de entrada (main)

`main()` arranca la interfaz. Inicializa `rcipy`, crea el `QApplication` y la ventana, y entra en el bucle de eventos de Qt. Al salir del bucle, destruye el nodo y cierra `rcipy` en un bloque `finally`.

7. Scripts de arranque y parada

Los cuatro scripts de `scripts/` orquestan el arranque y la parada en cada entorno. Todos activan primero el entorno virtual (`laric_env`).

- **start_simulation.sh:** Lanza, en orden y con esperas para que cada etapa registre sus tópicos, Gazebo (mundo `turtlebot3_house`, con el robot apareciendo en `x = -2.0`, `y = -3.0`, lejos de obstáculos), Nav2 + RViz (con el YAML de parámetros y el mapa, sobrescribible con la variable `MAP_PATH`), la HMI y el agente (la HMI antes, para que su publicador `/language_changed` exista primero). Cada proceso redirige su salida a `laric_logs/<marca>/`.
- **stop_simulation.sh:** Mata con `pkill` el agente, la HMI, los "ros2 launch", los contenedores de componentes, Nav2, `robot_state_publisher`, Gazebo (`gzserver/gzclient`) y RViz, y detiene el daemon de ROS 2.
- **start_real.sh:** Solo el lado del PC (HMI y agente). Exporta `ROS_DOMAIN_ID=0` y `ROS_AUTOMATIC_DISCOVERY_RANGE=SUBNET`, hace ping al robot (192.168.1.46) para avisar si no está accesible, e indica la URL de Foxglove. Nav2 corre a bordo del robot.
- **stop_real.sh:** Mata solo el agente y la HMI del PC y recuerda que la navegación del robot se detiene con "just stop" desde el ROSbot 3.

8. Parámetros de Nav2 (`config/laric_nav2_params.yaml`)

Este archivo configura el stack de Nav2 para la simulación y se pasa a `navigation2.launch.py` con `params_file`. Son, en su mayoría, parámetros estándar de Nav2 (AMCL, planificador, controlador y los behavior plugins `/spin`, `/drive_on_heading` y `/backup`), no código propio de LARIC, por lo que aquí solo se menciona su existencia. El robot real no usa este archivo: lleva su propia configuración a bordo (`rostop-autonomy`).

9. El fork de RAI y los errores en `nav2.py`

LARIC depende de un fork modificado de RAI (github.com/lourdesfll29-maker/rai). Durante la integración de la navegación se detectaron dos defectos en las herramientas de Nav2 de RAI. Ambos están en un único archivo: `tools/ros2/navigation/nav2.py`. Afectan a cómo RAI expone el resultado de una acción `/navigate_to_pose`, que LARIC consulta por sondeo (`GetNavigateToPoseResultTool`, véase 5.9) para saber si el robot ha llegado o ha fallado.

El problema de fondo es que el estado de una acción previa se cuela en la siguiente. Esto también afectaba a los movimientos primitivos (`/spin`, `/drive_on_heading`, `/backup`): tras una cancelación, el siguiente comando podía heredar el estado cancelado del anterior y abortarse a sí mismo. Pero ese caso se resuelve en el propio código de LARIC, reiniciando `abort_event` al inicio de cada comando (véase 5.4 y 5.5). El estado de la navegación, en cambio, lo mantiene RAI, así que hubo que corregirlo en el fork.

9.1. Estado de la navegación no reiniciado entre misiones

Las herramientas de navegación de RAI guardan el identificador, el feedback y el resultado de la acción en variables globales del módulo (`current_action_id`, `current_feedback`, `current_result`). El código original solo las rellena desde los callbacks del objetivo en curso, pero nunca las limpia. Por ello, tras la primera navegación, `GetNavigateToPoseResultTool` seguía devolviendo indefinidamente el estado terminal de la misión anterior, y la siguiente navegación se daba por concluida nada más empezar, al leer ese estado obsoleto.

- **Modificación del fork:** `NavigateToPoseTool._run` reinicia esas variables a `None` al comienzo de cada llamada, de modo que cada misión parte de un estado limpio.

```
# Añadido al inicio de NavigateToPoseTool._run (fork):
global current_action_id, current_feedback, current_result
current_action_id = None
current_feedback = None
current_result = None
# Sin esto, GetNavigateToPoseResultTool devuelve para siempre el
# estado terminal de la misión anterior tras la primera navegación.
```

Código 8. Reinicio del estado de la navegación (RAI).

9.2. GoalStatus descartado en el resultado

El result final de ROS 2 tiene dos partes: un payload (el mensaje Result de NavigateToPose, vacío en Nav2) y un GoalStatus (4 = SUCCEEDED, 5 = CANCELED, 6 = ABORTED). El RAI original devolvía el payload vacío (`str(current_result.result().result())`) y descartaba el GoalStatus. Así devolvía lo mismo tanto si el robot llegaba como si chocaba.

- **Modificación del fork:** Ahora se devuelve el GoalStatus real (`str(current_result.result().status())`). Con eso, NavigationTool lo compara con `STATUS_SUCCEEDED`, `STATUS_ABORTED` o `STATUS_CANCELED` y distingue una llegada de un fallo.

```
# GetNavigateToPoseResultTool._run
# Original (descarta el GoalStatus, devuelve el payload vacío):
# return str(current_result.result().result)
# Fork (devuelve el GoalStatus real: 4=SUCCEEDED, 5=CANCELED, 6=ABORTED):
return str(current_result.result().status)
```

Código 9. Obtención del GoalStatus (RAI).

Ambas correcciones son imprescindibles. Sin la primera, solo funcionaría la primera navegación de cada sesión. Sin la segunda, nunca se distinguiría una llegada de un fallo. Con el RAI oficial, la detección de llegada o fallo de la navegación no funcionaría.

10. Internacionalización: catálogos

Los textos visibles se extraen a una plantilla (`locales/laric.pot`), se traducen en los archivos `.po` (`es/LC_MESSAGES/laric.po` y `en/LC_MESSAGES/laric.po`) y se compilan a `.mo` binario con `msgfmt`, que es lo que `i18n.py` carga en tiempo de ejecución. Para añadir o cambiar un texto, los pasos son:

1. Envolver el texto en `_()`.
2. Regenerar la plantilla `.pot` (`xgettext`).
3. Actualizar los `.po` (`msgmerge`).
4. Traducir las cadenas nuevas.
5. Recompilar los `.mo` (`msgfmt`).

Las cadenas internas que el LLM evalúa (`SUCCESS`, `ACTION FAILED`, etc.) no se traducen: el prompt las reconoce por su prefijo en inglés.